EE 585: Probabilistic Robotics

Homework 2

Author: Çağdaş Güven
Due: November 19, 2025
This report presents the solutions, derivations, and analysis for Homework 2, covering Markov chains, motion models, robot dynamics, and sensor models.

Q1: Markov Weather Simulator

(a) Probability Calculation

The problem is to find the probability of the sequence (Day2=Cloudy, Day3=Cloudy, Day4=Rainy) given that Day 1 is Sunny. This is a Markov chain, so the probability of the sequence is the product of the individual transition probabilities.
The transition table is given as:

Today	**P(Sunny tomorrow)**	**P(Cloudy tomorrow)**	**P(Rainy tomorrow)**
Sunny	0.8	0.2	0.0
Cloudy	0.4	0.4	0.2
Rainy	0.2	0.6	0.2

The calculation is as follows:
P(Sequence) = P(D2=C | D1=S) * P(D3=C | D2=C) * P(D4=R | D3=C)
P(Sequence) = (0.2) * (0.4) * (0.2)
P(Sequence) = 0.016
The probability of this specific weather sequence is **1.6%**.

(b) Python Simulator

A Python simulator was written (submitted as generate_weather.py) to generate random weather sequences based on the Markov transition matrix. The core logic of the simulator is described in the following pseudocode:

function generate_weather(start_state, num_days):
  states = ["sunny", "cloudy", "rainy"]
  transitions = [[0.8, 0.2, 0.0],
  [0.4, 0.4, 0.2],
  [0.2, 0.6, 0.2]]

$current_{index} = index_of(start_state)$

```
sequence = [start_state]

for i from 1 to num_days:
    // Get the probability row for the current state
    probabilities = transitions[current_index]

    // Choose the next state based on these probabilities
    next_index = numpy.random.choice([0, 1, 2], p=probabilities)

    // Add to sequence and update state
    sequence.append(states[next_index])
```

$current_{index} = next_{index}$

```
return sequence
```

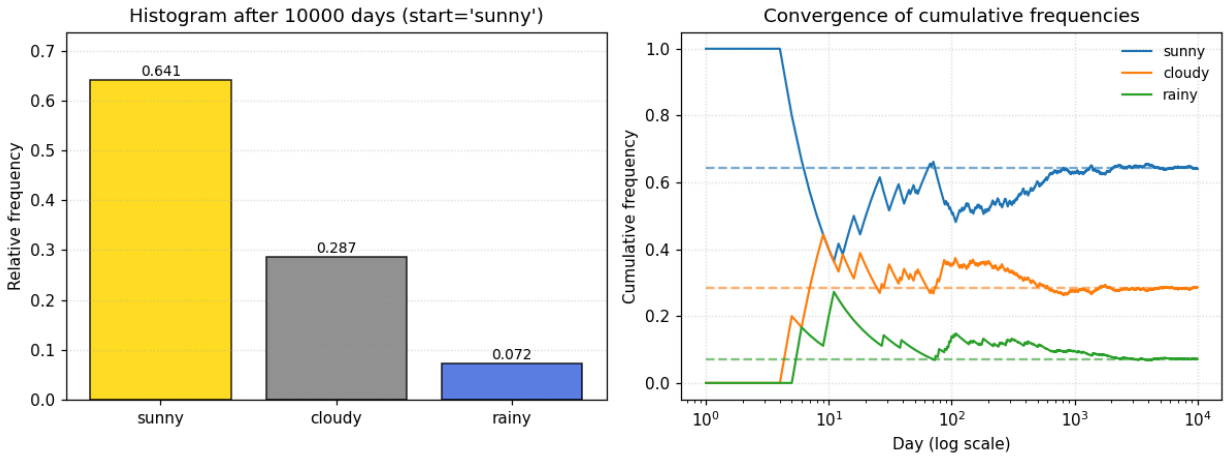

Figure 1: Histogram of weather states from a 10,000-day simulation.

Analysis: After running the simulation for a large number of days (e.g., N=10,000), the distribution of weather states converges to a stationary distribution, and the cumulative frequencies of each state stabilize. This distribution shows the long-term probability of any given day being sunny, cloudy, or rainy, regardless of the starting state.

Q2: Velocity Motion Model

This question involves implementing and analyzing the velocity motion model from the lecture slides (Page 34) and textbook (Chapter 5.3). The full Python scripts (*motion_model_ideal*.py and *motion_model_noisy*.py) are submitted separately.

(a) Ideal Model

The ideal, noise-free motion model calculates the robot's new pose (x', y', θ') from the current pose (x, y, theta) and the velocity commands (v, w) over a time step dt.
The kinematic equations for a non-zero rotational velocity (w != 0) are:

x' = x - (v/w)*sin(theta) + (v/w)sin(theta + wdt)
y' = y + (v/w)cos(theta) - (v/w)cos(theta + wdt)
theta' = theta + wdt

For the special case of straight-line motion (w = 0):

x' = x + vdt * cos(theta)
y' = y + vdt * sin(theta)
theta' = theta

The Python script q2a_ideal_model.py was used to simulate this ideal path. Figure 2 shows the resulting trajectory from an initial pose of (0, 0, 0) with v=1.0 and w=0.5.

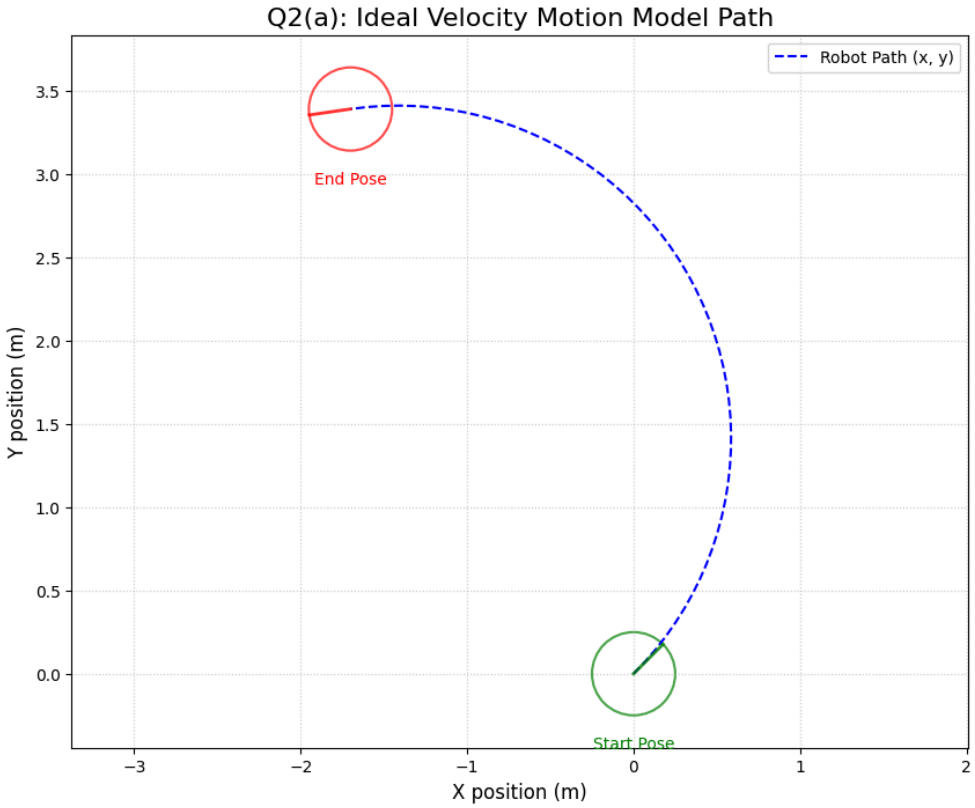

Figure 2: Simulated path of the ideal (noise-free) velocity motion model over 10 seconds, with poses plotted every 0.1 seconds.

(b) Noisy Model

The noisy model implements the sampling algorithm (sample_motion_model_velocity from Lecture Slides, Page 36) to simulate the effects of motion error. The simulation was run N=700 times from the same initial pose (0, 0, 0) for three different sets of noise parameters $(\alpha_1 through \alpha_6)$.

The plots in Figure 3 show the single starting pose (black robot) and the resulting cloud of 700 possible end poses (blue scatter plot), with the calculated mean of these end poses (red robot).

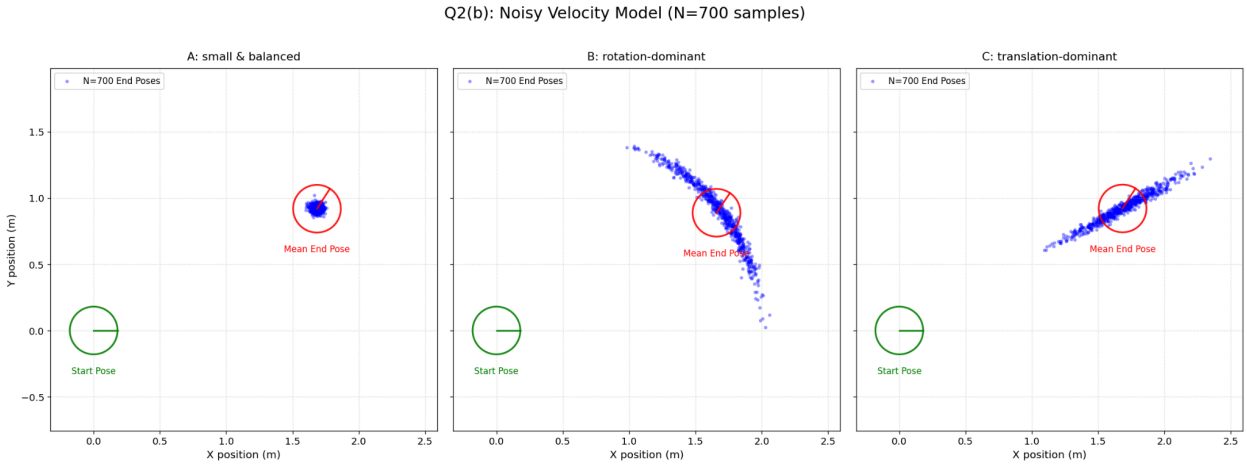

Figure 3: Final pose distributions for the noisy velocity motion model (N=700). Each plot uses different alpha noise parameters, starting from the same initial pose.

(c) Analysis of Noise Parameters

The shape of the sample cloud in Figure 3 is directly related to which noise parameters were dominant. The model introduces noise to the commanded velocities (v, w) and adds a final random drift gamma_hat.

- **Translational Noise** (α_1, α_2)**:** Controls variance in v_hat. Causes spread *along* the arc of motion.
- **Rotational Noise** (α_3, α_4)**:** Controls variance in w_hat. Causes spread *perpendicular* to the arc, making the "banana" shape wider.
- **Drift Noise** (α_5, α_6)**:** Controls variance in gamma_hat. This is a random rotation added *after* the path is complete, causing the samples to "fan out" from the end point.

1. Plot A: "Small, balanced noise"

- **Parameters:** $(\alpha_1, \ldots, \alpha_6)$ = (0.0001, 0.0001, 0.0001, 0.0001, 0.0001, 0.0001)
- **Analysis:** With all parameters small, all error sources are minimal. The resulting cloud is small and tightly clustered around the mean, representing a high-precision robot.

2. Plot B: "Large final rotation noise"

- **Parameters:** $(\alpha_1, ..., \alpha_6)$ = (0.0001, 0.0001, 0.02, 0.02, 0.02, 0.02)
- **Analysis:** Here, alpha_5 and alpha_6 are large, while alpha_1 through alpha_4 are small. This means the robot consistently follows the *same ideal path* (since v_hat is close to v and w_hat is close to w) but then performs a *large, random turn* at the very end. This is what creates the characteristic "fanning out" shape along an arc, where the final position is consistent but the final heading is highly uncertain.

3. Plot C: "Large motion noise"

- **Parameters:** $(\alpha_1, ..., \alpha_6)$ = (0.01, 0.01, 0.0001, 0.0001, 0.0001, 0.0001)
- **Analysis:** Here, alpha_1 through alpha_4 are large, while the final drift parameters are small. This means each of the 700 samples has very different v_hat and w_hat values from the start. Each sample follows a unique, erratic path. Some travel further, some shorter, some turn more, some less. This results in a large, diffuse cloud where both the final (x, y) position and orientation are highly variable.

Q3: Robot Dynamics (Textbook Ch. 5, Q1)

The question asks to solve Chapter 5, Question 1 from the textbook: "Derive the motion model for a one-dimensional robot... The state space is given by $\mathbf{x}_t = (x_t \ \dot{x}_t)^T$ ... The control u_t is an acceleration... $u_t = \ddot{x}_t$." We must also account for Gaussian noise in the acceleration.

This requires defining a state transition model $p(\mathbf{x}t|u_t, \mathbf{x}t-1)$ where the state includes both position x and velocity $\dot{x}$ (x_{dot}).

1. Define State and Control:

- State at time $t-1$: $x_{t-1} = \begin{pmatrix} x_{t-1} \\ \dot{x}_{t-1} \end{pmatrix}$
- State at time t: $x_t = \begin{pmatrix} x_t \\ \dot{x}_t \end{pmatrix}$
- Control at time t: $u_t = \ddot{x}_t$ (acceleration, held constant for Δt)

2. Ideal (Noise-Free) Dynamics:

Using standard physics, the velocity and position at time t are found by integrating the constant acceleration u_t over the interval Δt:

$\dot{x}t = \dot{x}t-1 + u_t \cdot \Delta t$

$$x_t = x_{t-1} + \dot{x}_{t-1} \cdot \Delta t + 0.5 \cdot u_t \cdot (\Delta t)^2$$

3. Matrix Form (Linear Model):

We can express this deterministic model in the standard linear state-space form $\mathbf{x}t = A\mathbf{x}t - 1 + Bu_t$:

$$\begin{pmatrix} x_t \\ \dot{x}_t \end{pmatrix} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_{t-1} \\ \dot{x}_{t-1} \end{pmatrix} + \begin{pmatrix} \frac{1}{2}(\Delta t)^2 \\ \Delta t \end{pmatrix} u_t$$

Here, $A = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix}$ and $B = \begin{pmatrix} \frac{1}{2}(\Delta t)^2 \\ \Delta t \end{pmatrix}$.

4. Adding Noise (Probabilistic Model):

The prompt specifies that the control (acceleration) is subject to Gaussian noise. We can also assume a more general process noise $\boldsymbol{\epsilon}_t$ that perturbs the final state, accounting for unmodeled effects like friction.

$$x_t = Ax_{t-1} + Bu_t + \epsilon_t$$

where $\boldsymbol{\epsilon}_t$ is a 2D Gaussian noise vector $\sim \mathcal{N}(\mathbf{0}, R_t)$, and R_t is the process noise covariance matrix.

5. Final Motion Model $p(\mathbf{x}t|u_t, \mathbf{x}t - 1)$:

The resulting state $\mathbf{x}_t$ is a Gaussian random variable. The motion model is the PDF of this Gaussian:

$$p(\mathbf{x}t|u_t, \mathbf{x}t - 1) = \mathcal{N}(\mathbf{x}_t; \boldsymbol{\mu}_t, \Sigma_t)$$

Where the mean vector $\boldsymbol{\mu}_t$ and covariance matrix Σ_t are:

- $\boldsymbol{\mu}t = A\mathbf{x}t - 1 + Bu_t = \left[x_{t-1} + \dot{x}t - 1\Delta t + 0.5u_t(\Delta t)^2 \ \dot{x}t - 1 + u_t\Delta t\right]$

- $\Sigma_t = R_t$ (This is the process noise covariance matrix. A simple model, assuming noise in acceleration u_t with variance σ_u^2, would be $\Sigma_t = BB^T\sigma_u^2$.)

This is a linear-Gaussian model, which could be directly used in a Kalman Filter.

Q4: Continuous Camera Sensor Model

This question asks for the derivation and description of a 2D planar pinhole camera model.

(a) The Geometric (Pinhole) Model

The 2D planar pinhole camera model provides the "ideal" (deterministic) geometry for projecting a 2D world landmark onto a 1D image.

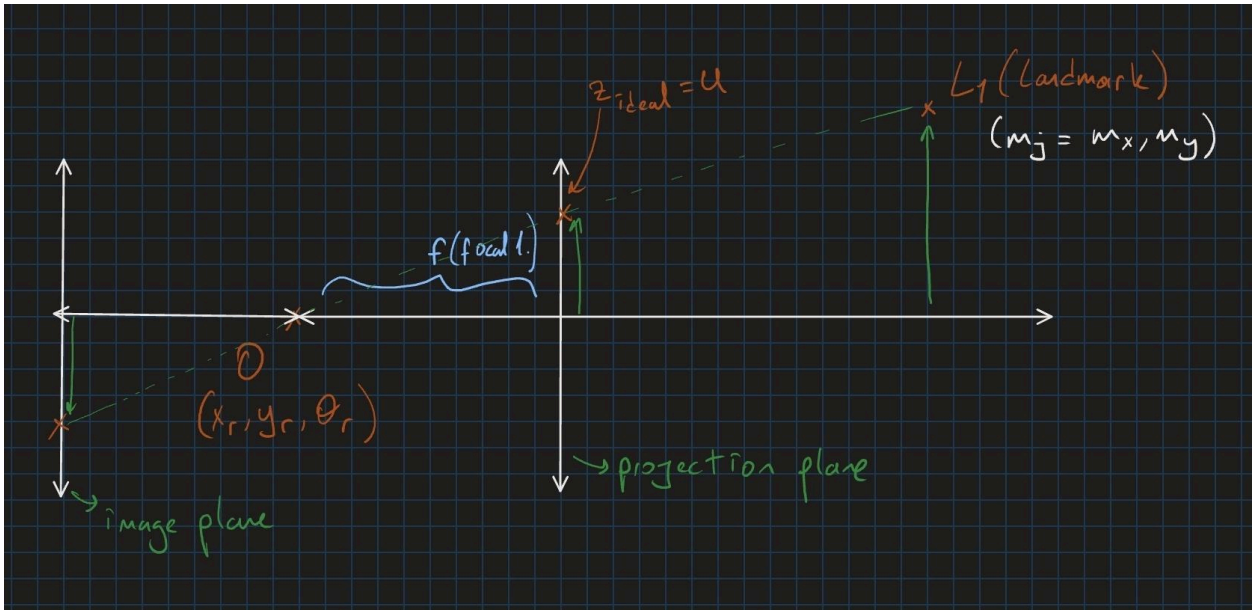

Figure 3: The 2D planar pinhole camera model, showing the relationship between the robot's pose x, *the landmark* $L1$, *and the ideal projection* .

As shown in Figure 4, the model parameters are:

- **Robot Pose (x_r):** This is the state we want to estimate, $x_r = (x_r, y_r, \theta_r)$. The optical center 'O' is at (x_r, y_r) and the robot's local x-axis (its "nose") points in the direction θ_r.
- **Landmark Position (m_j):}** This is the "map," a known point $m_j = (m_x, m_y)$.
- **Focal Length (f):** A fixed, known parameter of the camera. It is the distance from the optical center 'O' to the image plane.
- **Image Plane:** A 1D line segment.
- **Ideal Measurement (z_{ideal}):}** The 1D coordinate on the image plane where the

landmark m_j is projected.

Assumptions:

1. The landmark m_j is a single point with no size or orientation.
2. We use the "virtual image plane" convention (as shown in the drawing), placing the image plane *in front* of the optical center 'O'. This is mathematically simpler and results in a non-inverted (upright) projection.
3. The 1D image plane is perpendicular to the robot's local x-axis.

(b) The Noise Model

The homework specifies the noise model: "Gaussian additive white noise on the continuous pixel coordinates." This means the *actual* measurement z is the *ideal* projection z_ideal plus noise:

$z = z_{ideal} + \epsilon$where epsilon $N(0, sigma^2)$

This defines the forward probability $p(z|x, m)$:

$$p(z|x, m) = \mathcal{N}(z; z_{ideal}, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\frac{(z - z_{ideal})^2}{\sigma^2}\right)$$

where z_{ideal} is a function of x and m.

(c) Procedure

1.

For Q4(d), Forward Model: The goal is to compute $p(z | x, m)$. The procedure is:

1. Given $x = (x_r, y_r, \theta_r)$ and $m_j = (m_x, m_y)$.
2. Convert the landmark's global position m_j into the robot's local coordinate frame $(L_{\text{local}_x}, L_{\text{local}_y})$ using rotation and translation.
3. Use the geometry of similar triangles to derive the deterministic projection $z_{\text{ideal}} = h(x, m_j)$.
4. The final probability $p(z|x, m)$ is the Gaussian PDF from Q4(b), centered at this z_{ideal}.

2.

For Q4(e), Sampling Model: The goal is to sample from $p(x|z, m)$. This requires inverting the model.

1. The 1D measurement z only provides information about *one* degree of freedom: the bearing to the landmark. It provides *no* information about distance.
2. This is an ill-posed problem (1 measurement, 3 unknowns: x_r, y_r, θ_r).
3. The procedure will be to sample a possible pose x that is consistent with the measurement z, which involves "inventing" the missing information (distance and position on the bearing circle).

(d) Derivation: algorithm_camera_model(z, x, m)}

This function computes $p(z|x,m)$. Let $x=(x_r,y_r,\theta_r)$ and $m_j=(m_x,m_y)$.

Step 1: Convert landmark to robot's local frame.

$$\begin{pmatrix} L_{local\_x} \\ L_{local\_y} \end{pmatrix} = \begin{pmatrix} \cos(\theta_r) & \sin(\theta_r) \\ -\sin(\theta_r) & \cos(\theta_r) \end{pmatrix} \begin{pmatrix} m_x - x_r \\ m_y - y_r \end{pmatrix}$$

Step 2: Derive $z\mathrm{ideal}$ using similar triangles.

From the geometry in Figure 4, we have the relationship:

$$\frac{z_{ideal}}{f} = \frac{L_{local\_y}}{L_{local\_x}} \implies z_{ideal} = f\frac{L_{local\_y}}{L_{local\_x}}$$

This gives us the final deterministic function $h(x,m_j)$:

Step 3: Present the algorithm.

The function computes the probability $p(z|x,m_j)$ by plugging the z_{ideal} from Step 2 into the noise model from Q4(b).

function algorithm_camera_model(z, x_robot, m_landmark, f, sigma):
// x_robot = (xr, yr, thr)
// m_landmark = (mx, my)

// Step 1: Convert to local frame
dx = mx - xr
dy = my - yr
cos_th = cos(thr)
sin_th = sin(thr)

L_local_x = dx * cos_th + dy * sin_th
L_local_y = -dx * sin_th + dy * cos_th

// Step 2: Check for validity (landmark must be in front of camera)
if L_local_x <= 0:
return 0.0 // Or a very small probability

// Step 3: Calculate ideal projection
z_ideal = f * (L_local_y / L_local_x)

// Step 4: Calculate probability using Gaussian PDF
variance = sigma * sigma
exponent = -0.5 * (z - z_ideal)*(z - z_ideal) / variance
probability = (1.0 / (sigma * sqrt(2*PI))) * exp(exponent)

return probability

(e) Derivation: sample_robotpose_camera_model(z, m_j, ...)

This function must sample a pose x_r from $p(x_r|z, m_j)$. A 1D measurement z only provides information on the *local bearing* α_{local}. This leads to the fundamental constraint equation:

$$\text{atan2}(m_y - y_r, m_x - x_r) = \theta_r + \text{atan2}(z_{\text{ideal}}, f)$$

This single equation has three unknowns (x_r, y_r, θ_r).

Answers to Questions:

- **"Would this function be able to sample absolute robot poses?"** No. A single 1D measurement provides only one constraint (bearing) for a 3D pose. It gives no information about the distance to the landmark, so the robot's (x_r, y_r) The position is ambiguous.
- **"If not, what can you sample?"** We can sample a pose x_r from the 2D *surface* of possible poses consistent with the measurement. More practically, we can sample the robot's orientation θ_r *given* an assumed position (x_r, y_r), or sample a position (x_r, y_r) *given* an assumed orientation θ_r.

Pseudocode for a Simplified Sampler:

This sampler "invents" a plausible distance and direction from the landmark to place the robot, then solves for the robot's orientation θ_r.

function sample_robotpose_camera_model(z, m_landmark, f, sigma, rng):
  // m_landmark = (mx, my)

  // Step 1: Sample from the noise model to get a bearing
  z_sample = z + rng.normal(0, sigma)

  // Step 2: Convert the noisy pixel z_sample to a local bearing alpha_local

```
alpha_local_sample = atan(z_sample / f)

// Step 3: Invent the missing 2 DoF: distance and direction from landmark
sampled_dist = rng.uniform(1.0, 10.0) // e.g., robot is 1-10m away
sampled_angle = rng.uniform(0, 2 * PI)  // e.g., robot is anywhere around landmark

xr_sample = mx + sampled_dist * cos(sampled_angle)
yr_sample = my + sampled_dist * sin(sampled_angle)

// Step 4: Solve for the 3rd DoF (theta_r) using the constraint
world_bearing = atan2(my - yr_sample, mx - xr_sample)
thr_sample = world_bearing - alpha_local_sample

// Normalize angle
thr_sample = (thr_sample + PI) % (2 * PI) - PI

return (xr_sample, yr_sample, thr_sample)
```

This algorithm correctly samples a pose, but it has to "invent" the distance and position (Step 3), which demonstrates that the measurement z alone is not sufficient to determine an absolute pose.

(f) Discussion

The derivations for Q4 reveal several major difficulties, which align with the lecturer's in-class explanation. The core issues are (1) the problem of data association, and (2) the asymmetric, non-linear nature of the 1D-to-3D mapping.

1. Data Association (The "Known Identity" Problem): The prompt for Q4(e) simplifies a major real-world challenge by stating, "Assume a landmark with known location, identity and color."

- **Without this assumption:** A robot seeing a "blip" at pixel z would face the full **data association problem**: "Is this Landmark 1, Landmark 2, or just a random object (a person)?" It would have to calculate a probability for all possibilities.
- **With this assumption:** We are allowed to skip this. We are told "this blip *is* Landmark 1 (m_1)." This simplification is what allows our functions to take a specific m_{landmark} as input, rather than the entire map m.

2. The 1D-to-3D Problem (The Core Difficulty): The central challenge is that we are trying to find a **3D pose** $x = (x_r, y_r, \theta_r)$ using only a **1D measurement** z.

As the derivation in (a) showed, the 1D pixel z (after noise) boils down to a single geometric

constraint: the local bearing α_{local}.

This gives us only **one equation**:

world_bearing = robot_orientation + local_bearing
$\text{atan2}(m_y - y_r, m_x - x_r) = \theta_r + \alpha_{\text{local}}$

It is mathematically impossible to find 3 unique unknowns (x_r, y_r, θ_r) from only 1 equation.

3. The "Uncertainty Cone" : This 1-equation-3-unknowns problem directly explains the lecturer's comments and the answers to Q4(e):

- The set of *all possible poses* that satisfy this single equation is not a point. It is a 2D surface in the 3D pose space.
- The Gaussian noise on z makes the α_{local} term fuzzy, which "smears" this surface of possible poses into a 3D volume, which the lecturer called the "uncertainty cone".
- This is why the answer to "Can you sample absolute poses?" is **No**.
- This is why the answer to "What can you sample?" is **A pose x from this 2D surface (or 3D "cone") of possibilities.**

This reveals the fundamental **asymmetry** of the model:

- **Forward Model $p(z|x,m)$:** Easy. The physics are causal. We start with a known 3D pose, which uniquely determines a 1D projection z_{ideal}. We just add simple 1D Gaussian noise.
- **Inverse Model $p(x|z,m)$:** Hard. We start with a 1D Gaussian noise distribution on the pixel plane. When we back-project this simple 1D Gaussian through the non-linear pinhole geometry, it creates a complex, non-Gaussian "weird density" in the 3D pose space.

Therefore, a simple Gaussian is sufficient for the *sensor model* $p(z|x,m)$, but it is completely insufficient for representing the robot's *belief* $\text{bel}(x) = p(x|z,m)$ after a single measurement.